GRADUATION / TRAINING PROJECT

AI Research Assistant

An Intelligent Research Assistant Powered by RAG & Multi-LLM Architecture

Mustafa Ahmed
Faculty of Computer Science & Artificial Intelligence
Artificial Intelligence Department
Beni Suef National University

AGENDA

Presentation Outline

1 Problem Statement

2 Project Objectives

3 System Architecture

4 Technology Stack

5 Knowledge Base

6 Retrieval-Augmented Generation

7 LangGraph Agent & Tools

8 Memory & Multi-LLM

9 Live Demo

10 Challenges

11 Future Work

12 Conclusion

MOTIVATION

Why Did We Build This Project?

LLM Limitations

- Hallucination — confident but incorrect answers
- Outdated knowledge frozen at training time
- No access to private or local documents
- Difficult to verify generated information
- Cannot answer questions from local research papers

Our Solution

Build an AI Assistant capable of answering questions directly from a local research knowledge base using Retrieval-Augmented Generation (RAG).

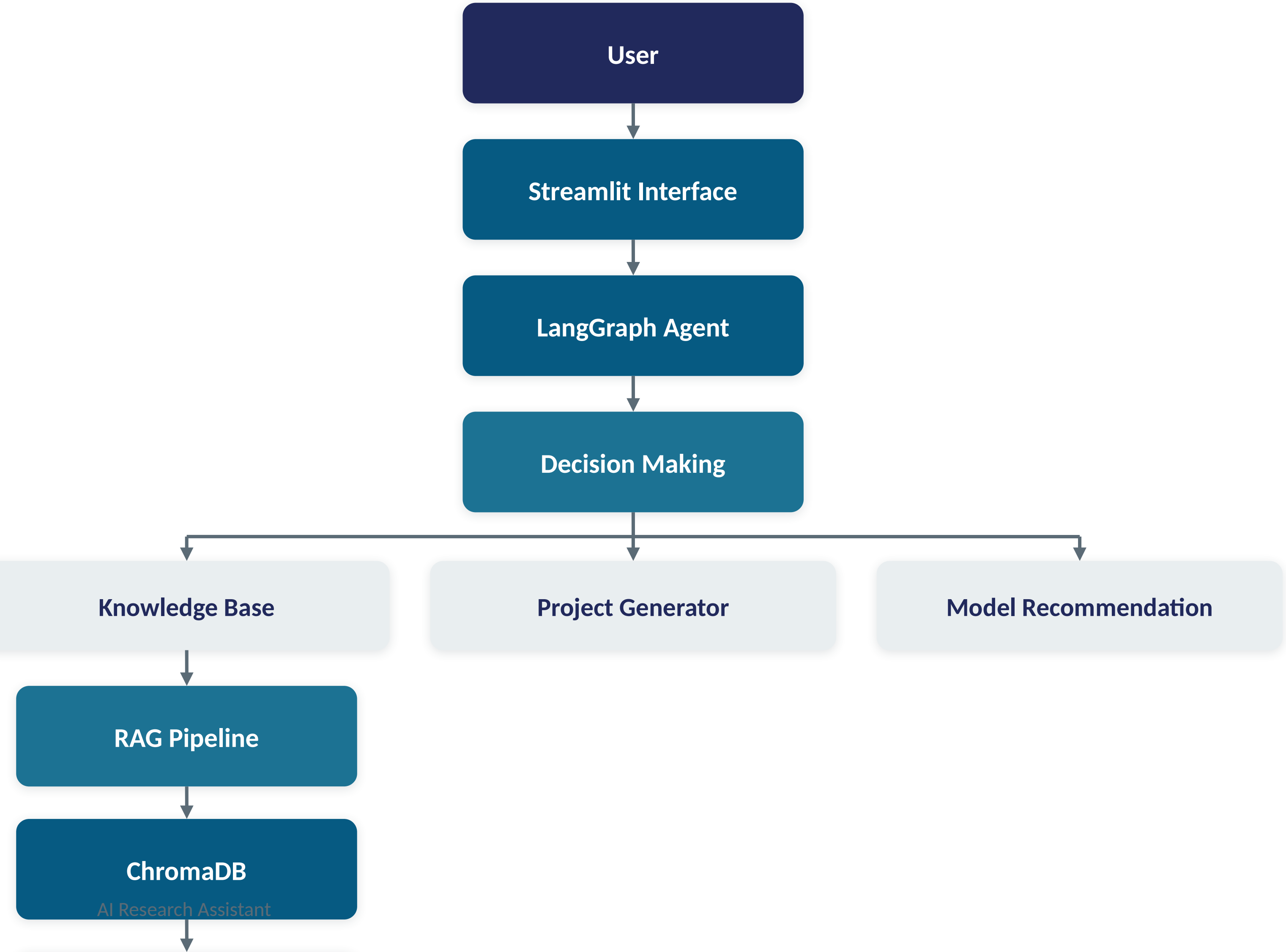
Grounded answers, verifiable sources, private knowledge — instead of relying on the model's memory alone.

GOALS

Main Objectives

- Build an AI-powered Research Assistant
- Reduce hallucinations using RAG
- Retrieve information from research papers
- Support multiple AI models
- Build an intelligent tool-based agent
- Create a modern conversational interface

Overall System Architecture



How it works

- The user interacts through the Streamlit interface.
- The LangGraph Agent decides which tool should handle the request.
- If the question needs document knowledge, it routes through the RAG pipeline.
- Otherwise, it calls the LLM directly.

STACK

Technology Stack

Frontend

- Streamlit

Backend

- Python

Frameworks

- LangChain
- LangGraph

Vector Database

- ChromaDB

Embedding Model

- sentence-transformers
- all-MiniLM-L6-v2

LLM Providers

- Google Gemini
- OpenRouter

Knowledge Base — Research Paper Collection

The assistant uses a local knowledge base containing research papers about Artificial Intelligence, indexed into ChromaDB for semantic retrieval.

- BERT
- Transformers
- Attention
- RAG
- CNN
- Vision Transformer
- GPT
- YOLO
- Deep Learning
- Computer Vision

ChromaDB

Vector database that stores and indexes document embeddings for fast semantic search.

- Indexes every research paper as vectors
- Runs similarity search on each query
- Returns the most relevant document chunks

Text Embedding & Vector Database

Document Chunk
Plain text passage

→

Embedding Model
all-MiniLM-L6-v2

→

ChromaDB
Stored as a vector

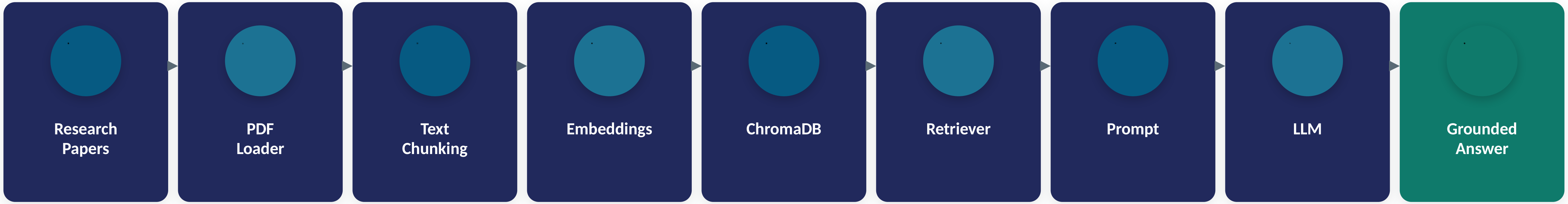
Why Embeddings?

- Enables semantic (meaning-based) search
- Fast retrieval even over large document sets
- High-quality similarity matching

ChromaDB Role

- Stores document embeddings persistently
- Performs semantic similarity search
- Returns the most relevant chunks to the agent

Retrieval-Augmented Generation Pipeline

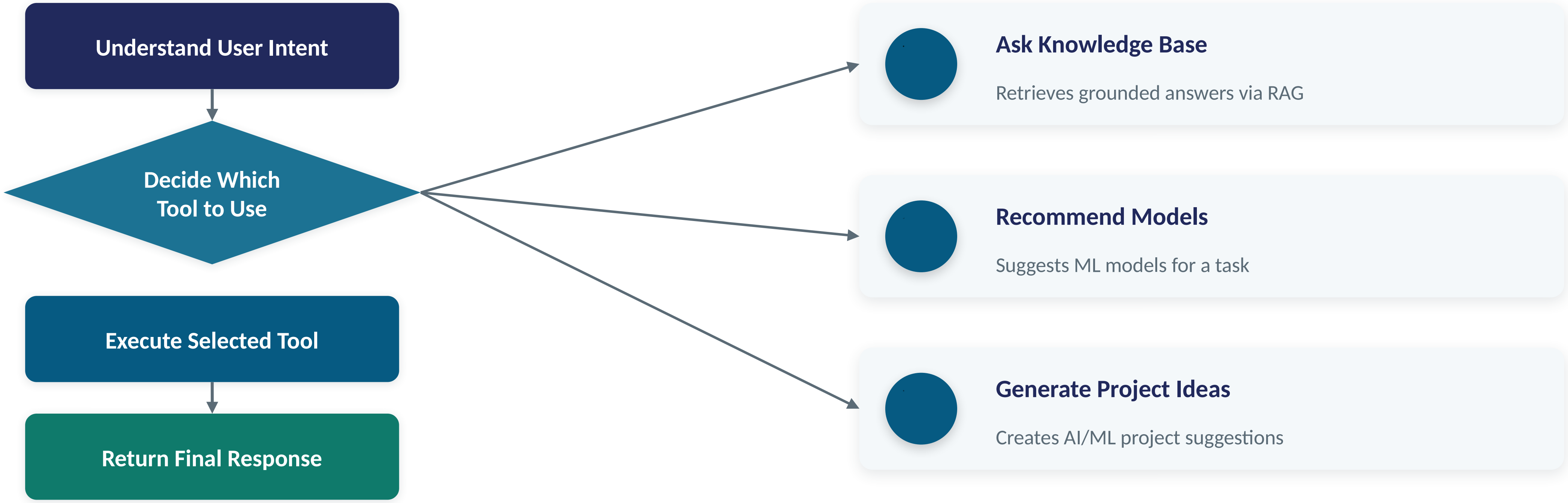


Why Retrieve Before Generating?

Instead of asking the LLM directly, the pipeline first retrieves the most relevant document chunks from ChromaDB, then sends both the question and the retrieved context to the model — producing an answer grounded in real research papers instead of the model's memory alone.

ORCHESTRATION

LangGraph Agent Workflow



The agent is modular and extensible — new tools can be added without changing the core graph.

Conversation Memory & Multi-LLM Strategy

Conversation Memory

The assistant remembers previous messages in the conversation.

- **Better follow-up answers**
- **Context-aware conversations**
- **More natural interactions**

Multi-LLM Strategy

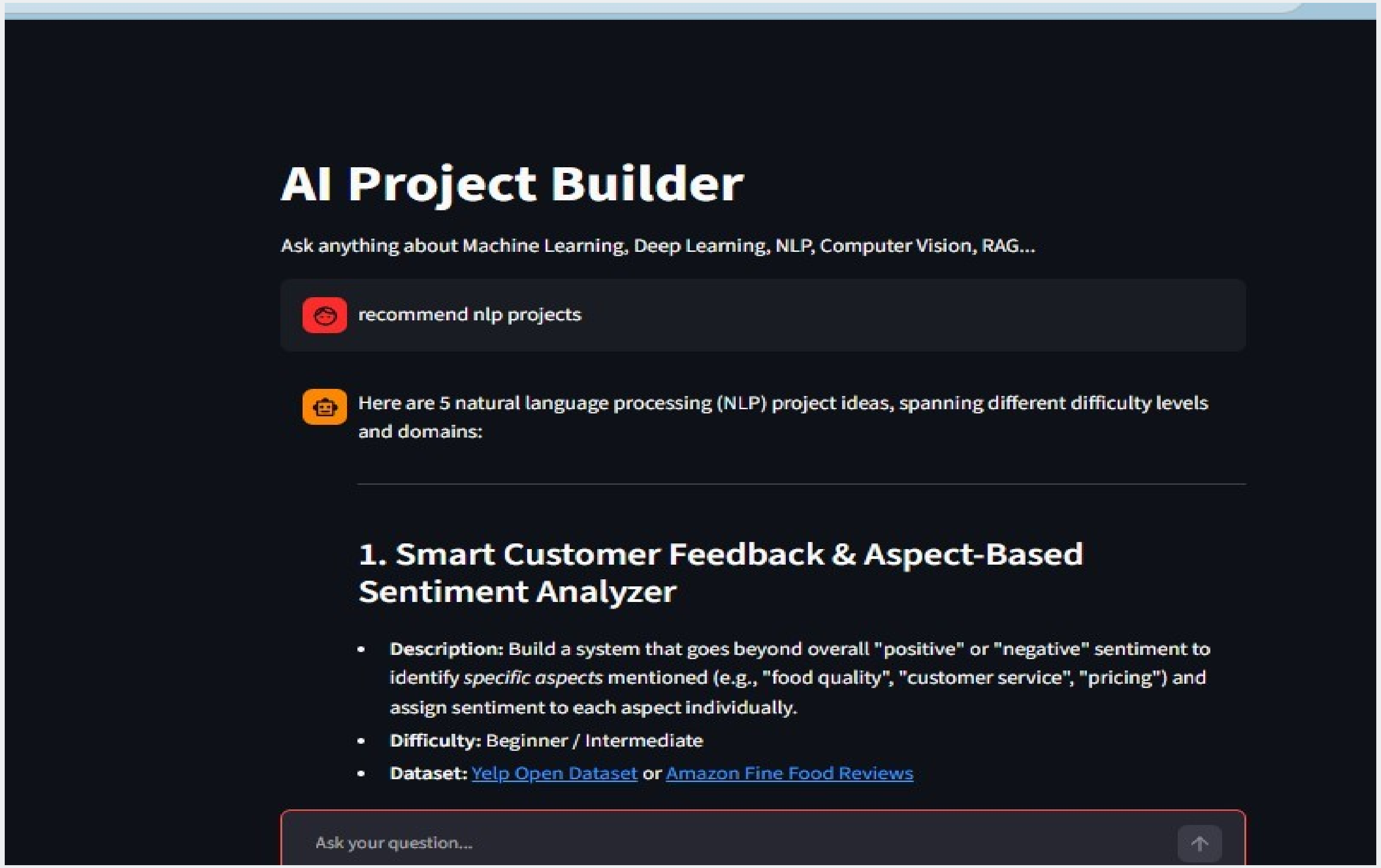
Google Gemini (Primary) → *if unavailable* → **OpenRouter (Fallback)**

- Higher availability
- Better reliability
- Automatic fallback — no manual switching

EXPERIENCE

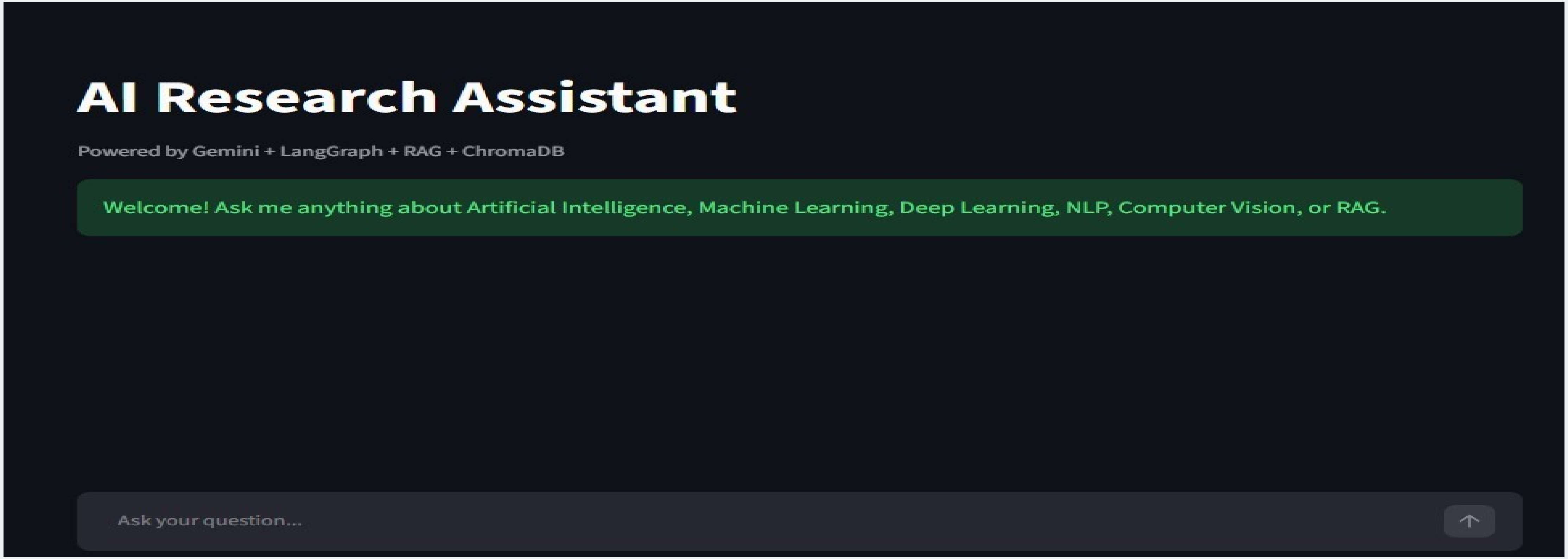
Streamlit Chat Interface

- Clean Chat UI
- Conversation History
- Sidebar Navigation
- Example Questions
- Clear Chat
- Error Handling



Designed for simplicity and ease of use.

Live Demo



Actual app response to "Explain BERT" — answer grounded in the local research knowledge base.

Try it: "Explain BERT" • "Explain RAG" • "Recommend models for sentiment analysis" • "Generate NLP project ideas"

Challenges & Solutions

Challenge	Solution
API quota limitations	Automatic fallback between Gemini and OpenRouter
Prompt engineering	Iteratively improved and tested prompts
Retriever quality	Better chunking strategy for cleaner context
Chunk size selection	Tuned chunk size for retrieval accuracy
Multiple LLM integration	Unified interface across providers
Conversation memory	Integrated memory into the agent state
GitHub secret protection	Secure API key management via .env

ROADMAP

Future Improvements

- Streaming Responses
- Source Citation
- Hybrid Search
- Web Search Integration
- PDF Upload
- Voice Assistant
- Authentication
- Cloud Deployment
- Docker Packaging
- Monitoring & Analytics

SUMMARY

What We Achieved

We successfully built an AI-powered Research Assistant that:

- **Uses Retrieval-Augmented Generation**
- **Retrieves knowledge from research papers**
- **Reduces hallucinations**
- **Supports multiple LLM providers**
- **Maintains conversation memory**
- **Provides intelligent tool-based reasoning**

This project demonstrates the integration of modern AI technologies — RAG, agentic orchestration, and multi-LLM reliability — into a practical, scalable application.

Thank You

Questions?

Mustafa Ahmed • AI Research Assistant • Beni Suef National University